DSA STUDY BLUEPRINT SERIES

120. Prims Algorithm Minimum Spanning Tree in Graph

Greedy Minimum Spanning Tree Construction (Prim's)

Section 10 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Prim's:** Build MST greedily starting from an arbitrary vertex.
- Select the cheapest adjacent edge using a min-heap.

Memory & Execution Blueprint

Active MST vertices set • add lowest adjacent edge weight to MST total.

C++ Implementation Reference

Standard code layout and syntax

```
int primMST(int V, vector>>& adj) {
    priority_queue, vector>, greater>> pq;
    vector inMST(V, false);
    pq.push({0, 0}); int mst_weight = 0;
    while (!pq.empty()) {
        int w = pq.top().first, u = pq.top().second; pq.pop();
        if (inMST[u]) continue;
        inMST[u] = true; mst_weight += w;
        for (auto edge : adj[u]) {
            if (!inMST[edge.first]) pq.push({edge.second, edge.first});
        }
    }
    return mst_weight;
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Priority Queue	O((V + E) log V) time, O(V) space

Key Practices & Gotchas

- **Important:** Avoid adding duplicate paths by verifying that vertex is not already in the MST.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace Dijkstra distance relaxation updates on active vertices:

Vertex popped	Adjacent edge checked	Current distance value	New path calculation	Relaxation action
U (Dist = 2)	U → V (Weight = 3)	V (Dist = 7)	New Dist = 2 + 3 = 5	Relax: Update V (Dist = 5)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Shortest Path choices:** BFS finds unweighted short paths; Dijkstra handles positive weights; Bellman-Ford resolves negative edges.
- **Spanning Trees:** Prim's builds MST from a start node; Kruskal's sorts all edges using disjoint sets.
- **Related LeetCode Problems:** #200 Number of Islands, #743 Network Delay Time, #787 Cheapest Flights Within K Stops.